Multivariate statistics II: PCR and PLSR

Solve Sæbø

Mar 18 2025

Topics for this lecture

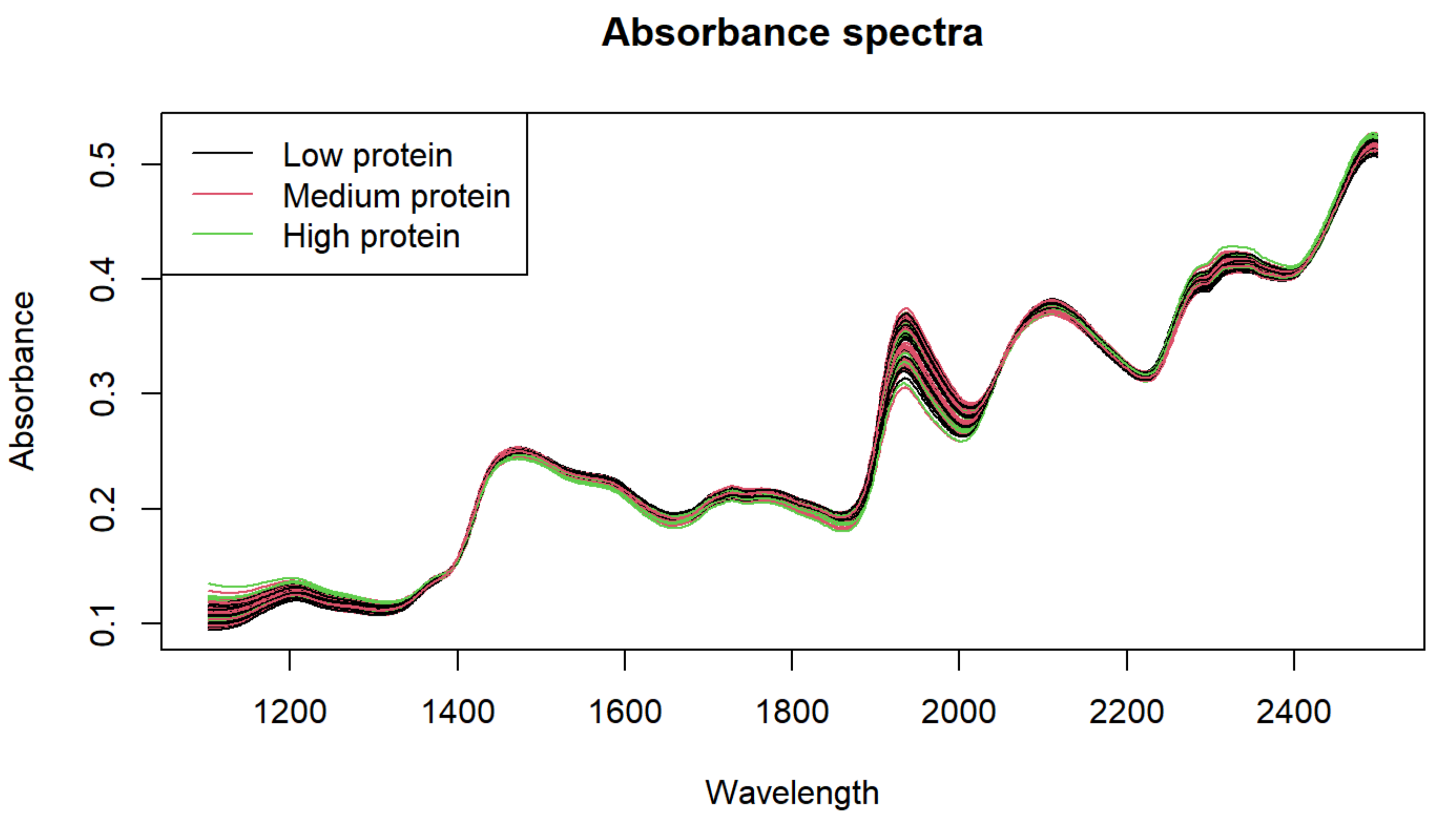
- Motivation for dimension reduction in regression
- Principal Components regression (PCR)
- Partial Least Squares Regression (PLSR)

Motivation for dimension reduction in regression

- The ordinary least squares estimator is much used since:
  1. It is an unbiased estimator
  2. Among all unbiased estimators it has minimum variance
- However, it has some problems:
  1. Requires $n > p$ (more observations than variables)
  2. May suffer from inflated variances when $n \sim p$, or…
  3. …if the predictors are highly collinear

Example - Chemometrics/Spectroscopy

- Near Infra Red (NIR)-spectroscopy absorbance spectra.
- $p = 700$ wavelenghts for $n = 100$ samples.



OLS under multi-collinearity, I

- Regression with one wavelength, w1522

```
##
## Call:
## lm(formula = Absorbance ~ NIR.w1522, data = NIRdata)
##
## Residuals:
##      Min       1Q   Median       3Q      Max
## -1.68013 -0.52481 -0.03636  0.49450  1.56003
##
## Coefficients:
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)    38.77       7.48   5.184 1.17e-06 ***
## NIR.w1522    -132.94      31.88  -4.169 6.60e-05 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.7131 on 98 degrees of freedom
## Multiple R-squared:  0.1507, Adjusted R-squared:  0.142
## F-statistic: 17.38 on 1 and 98 DF,  p-value: 6.598e-05
```

OLS under multi-collinearity, II

- Regression with two highly correlated predictors

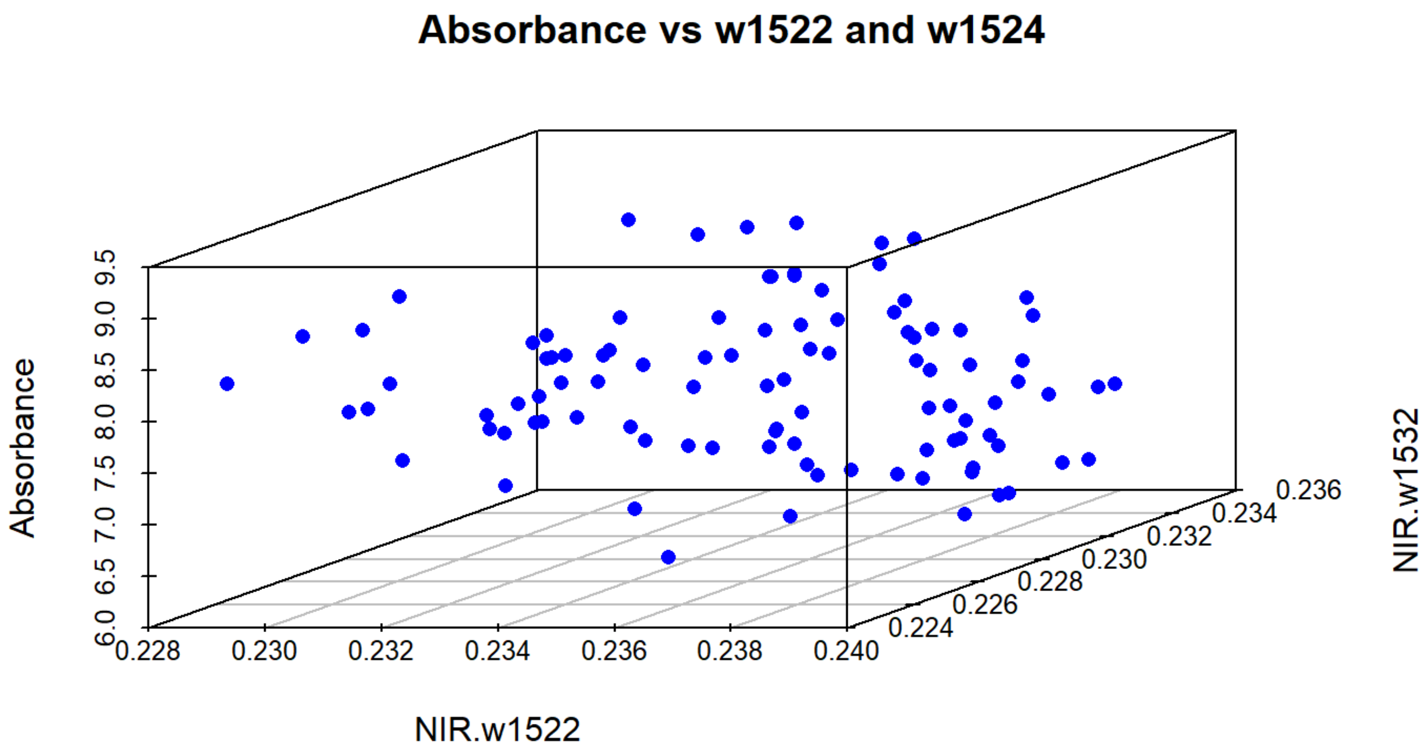
```
##
## Call:
## lm(formula = Absorbance ~ NIR.w1522 + NIR.w1532, data = NIRdata)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -1.3721 -0.4781 -0.1079  0.4943  1.5716
##
## Coefficients:
##              Estimate Std. Error t value Pr(>|t|)
## (Intercept)    54.904      8.701   6.310 8.34e-09 ***
## NIR.w1522   -2052.372    592.367  -3.465 0.000792 ***
## NIR.w1532    1878.503    578.968   3.245 0.001614 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 0.6808 on 97 degrees of freedom
## Multiple R-squared:  0.2338, Adjusted R-squared:  0.218
## F-statistic:  14.8 on 2 and 97 DF,  p-value: 2.456e-06
```

A graphical view of multicollinearity

- A dynamic 3D-plot of wavelengths 1522 and 1524 vs protein (Y) using the `rgl` package. Try yourself!

```
library(rgl)
attach(NIRdata)
plot3d(NIR.w1522, NIR.w1532, Absorbance, col= "blue", radius=0.05, type="s", xlab="w15
detach(NIRdata)
```

3D plot


Absorbance vs w1522 and w1524
Absorbance
6.0 6.5 7.0 7.5 8.0 8.5 9.0 9.5
0.228 0.230 0.232 0.234 0.236 0.238 0.240
NIR.w1522
0.224 0.226 0.228 0.230 0.232 0.234 0.236
NIR.w1532

OLS and overfitting

- Pick a random set of 100 variables as predictors.
- Fit a linear model
- Compute the R^2 and adjusted R^2

```
vars <- sample(2:701, 50, replace=FALSE)
fmla <- as.formula(paste("Absorbance ~ ", paste(names(NIRdata)[vars], collapse= "+")))
lmfit3 <- lm(fmla, data=NIRdata)
res <- summary(lmfit3)
res$r.squared
res$adj.r.squared
```

```
## [1] 0.930381
```

```
## [1] 0.8593411
```

How to handle these problems

- Forward variable selection in OLS? (data reduction)
- Use alternative estimator, e.g. PCR or PLSR (data reduction to ‘principal components’)

PCA of NIR data

- Let's return to the 3D plot from above
- Locate the principal components
- Discuss the variances of the two principal components
- How many PC's are needed to capture 99% of the variability in the entire NIR data?

PCR

- In PCR the principal components (PCs) replace the original X-variables as predictors in regression.
- Let a be the number of PCs we choose to use (e.g. explaining 99% X-variability)
- Typically we use $a << k$ components in the model.

The model with a components:

$$Y = \alpha_0 + \alpha_1 Z_1 + \alpha_2 Z_2 \cdots + \alpha_a Z_a + \epsilon$$

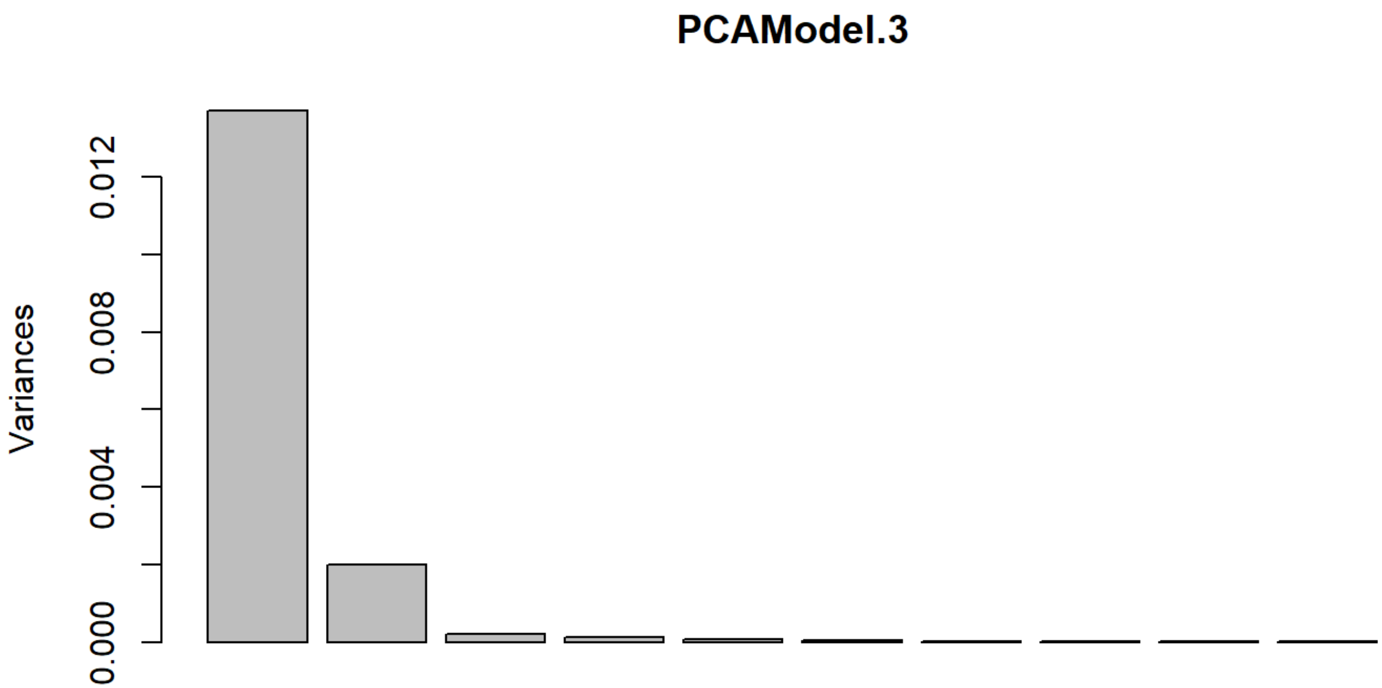
- The α's are estimated by the Ordinary Least Squares method.

Example, NIR data

- Run PCA on the Wavelength data, check results, scree plot

```
pcafmla <- as.formula(paste("~ ", paste(colnames(NIRdata)[2:600], collapse= "+")))
PCAModel.3 <- prcomp(pcafmla, scale.= FALSE, data=NIRdata)
screeplot(PCAModel.3)
```

The Scree plot



Saving some principal components as extra variables to the data

```
PCAdata <<- within(NIRdata, {
  PC5 <- PCAModel.3$x[,5]
  PC4 <- PCAModel.3$x[,4]
  PC3 <- PCAModel.3$x[,3]
  PC2 <- PCAModel.3$x[,2]
  PC1 <- PCAModel.3$x[,1]
})
```

Regression model, PC1-PC5

```
LinearModel.4 <- lm(Y ~ PC1 + PC2 + PC3 + PC4 + PC5, data=PCAdata)
sumfit <- summary(LinearModel.4)
round(sumfit$coefficients,3)
```

```
##             Estimate Std. Error t value Pr(>|t|)
## (Intercept)    7.588      0.052 145.807    0.000
## PC1            0.151      0.447   0.337    0.737
## PC2            4.061      1.171   3.468    0.001
## PC3           30.149      3.680   8.192    0.000
## PC4            3.063      4.520   0.678    0.500
## PC5           40.787      6.224   6.553    0.000
```

- Note that PC1 is not significant

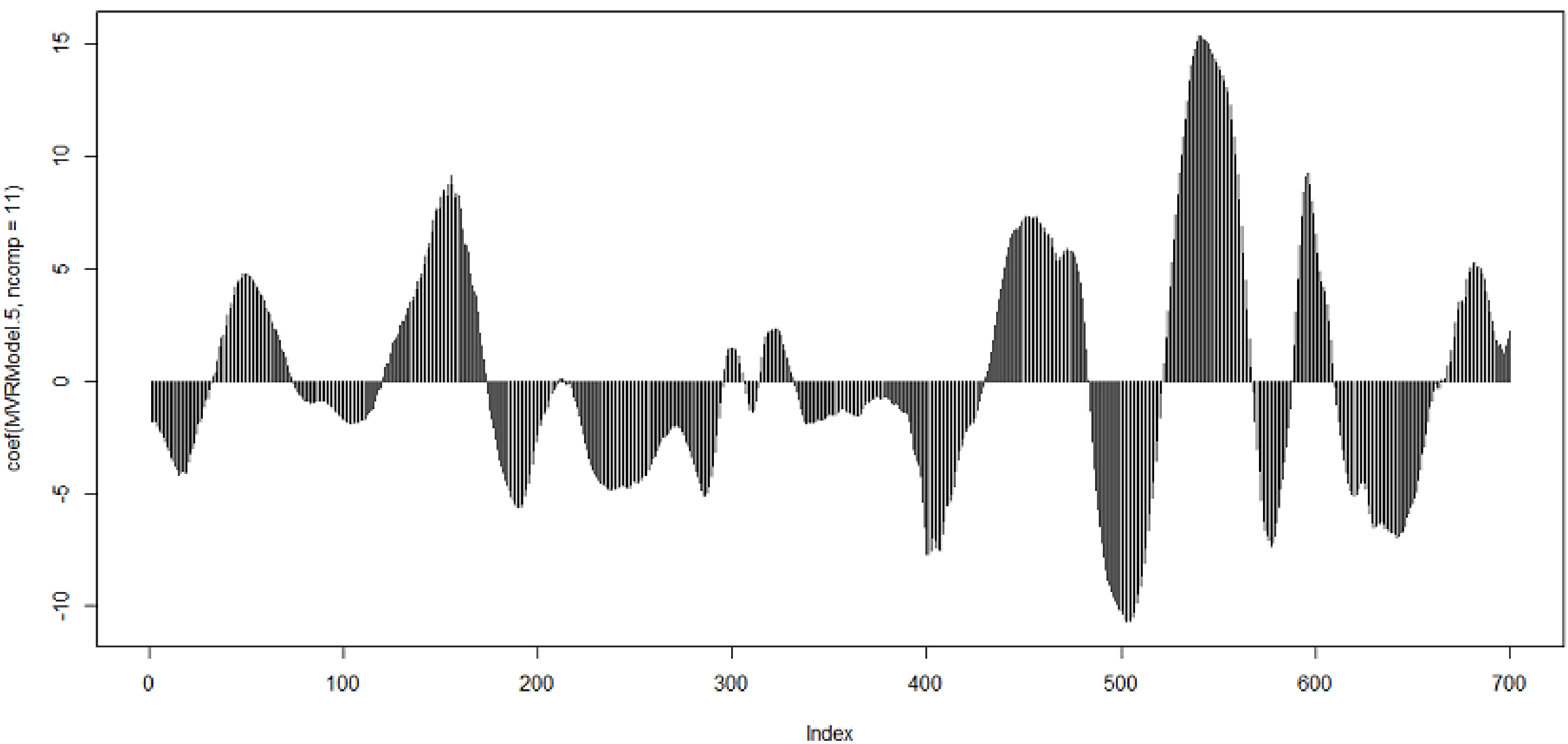
Running the PCR in RStudio with the pls package

- After estimation of the α's the regression coefficients for the *original* variables may be computed by a reverse transformation.
- These are easiest computed using the `pcr` function in the `pls` package.

```
library(pls)
pcrfit <- pcr(Absorbance ~ ., data=NIRdata, scale=FALSE)
summary(pcrfit)
coefs.pc11 <- coef(pcrfit, 11)
```

The regression coefficients plotted

- Here are the coefficients based on 11 PC's



Better to be presisly wrong…

…than approximately right

The PCR-estimates for the model with original predictors has typically these properties:

- They are biased if $a < k$ (but being closer to zero than OLS)
- Have smaller variances than the least squares estimates.

Prediction error may be smaller for PCR than OLS model since:

Expected squared prediction error =

σ^2 + var(estimates) + bias(estimates)^2

Choosing the number of components, a

- Cross-validation. As Leave-one-out (LOO-CV), or K-fold.
- Root Mean Squared Error of Prediction (RMSEP)

$$RMSEP_{LOO-CV} = \sqrt{\frac{1}{n}\sum_{i=n}^{n}(y_i - \hat{y_{(i)}})^2}$$

where $\hat{y_{(i)}}$ is a prediction of y_i based on a data set where observation i has been left out.

- Compare RMSEP for $a = 1, 2, \ldots$ and choose a simple, but good predictor.

LOO-CV example - R-code

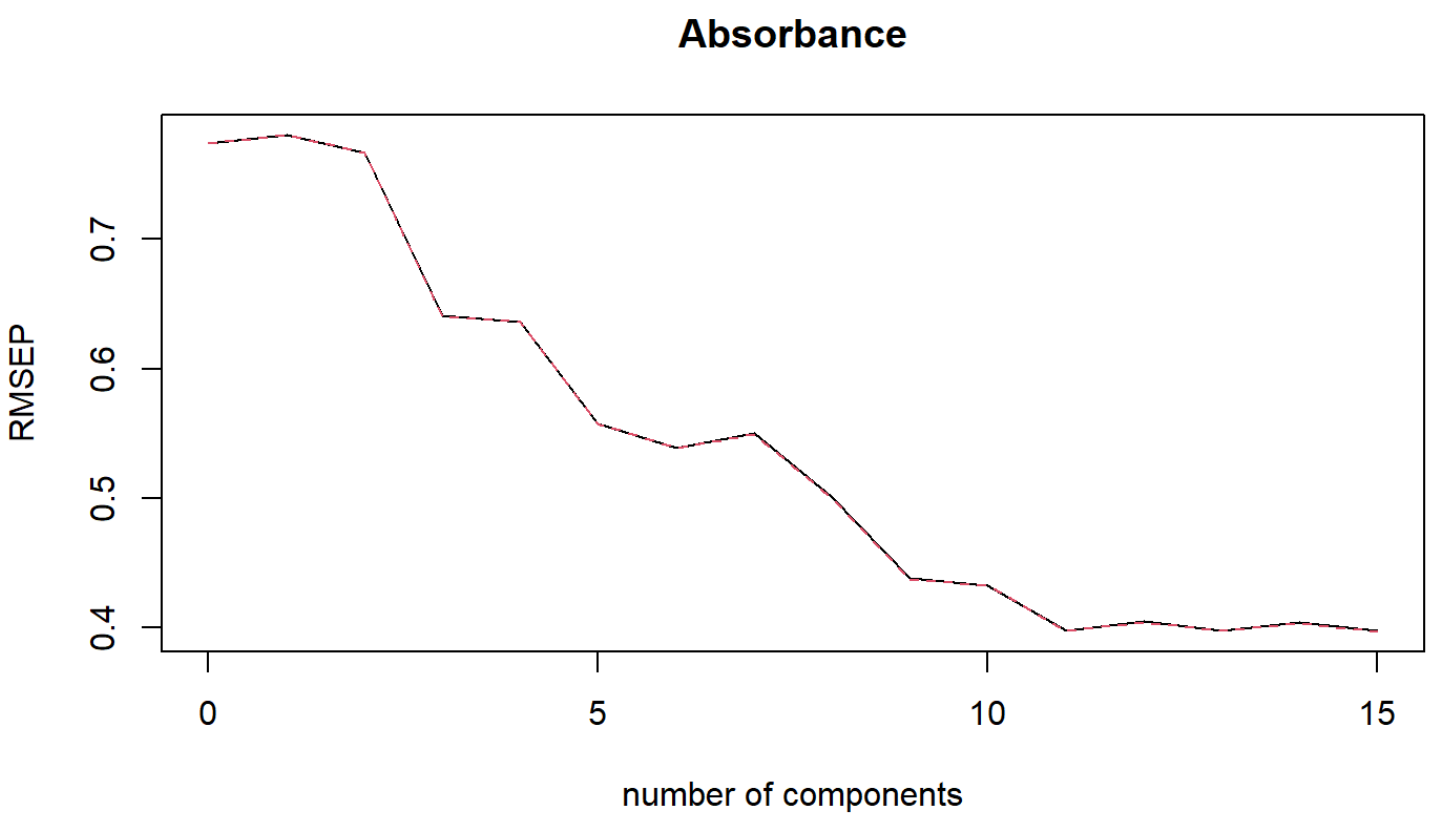
```
MVRModel.7 <- pcr(Absorbance ~ ., data=NIRdata, scale=FALSE, ncomp=15, validation="LOO
summary(MVRModel.7)
```

Summary with RMSEP-values

```
## Data:    X dimension: 100 700
##  Y dimension: 100 1
## Fit method: svdpc
## Number of components considered: 15
##
## VALIDATION: RMSEP
## Cross-validated using 100 leave-one-out segments.
##        (Intercept)  1 comps  2 comps  3 comps  4 comps  5 comps  6 comps
## CV          0.7737   0.7805   0.7668   0.6406   0.6364   0.5576   0.5386
## adjCV       0.7737   0.7804   0.7667   0.6401   0.6363   0.5575   0.5385
##        7 comps  8 comps  9 comps  10 comps  11 comps  12 comps  13 comps
## CV      0.5499   0.5005   0.4381    0.4328    0.3978    0.4046    0.3979
## adjCV   0.5497   0.5003   0.4375    0.4326    0.3975    0.4044    0.3976
##        14 comps  15 comps
## CV       0.4041    0.3974
## adjCV    0.4038    0.3971
##
## TRAINING: % variance explained
##              1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X           84.80628   96.599    97.82    98.86    99.44    99.66    99.74
## Absorbance   0.09501    5.334    35.27    37.55    52.14    56.57    56.74
##             8 comps  9 comps  10 comps  11 comps  12 comps  13 comps  14 comps
```

Plotting RMSEP

```
plot(RMSEP(MVRModel.7))
```



Partial Least Squares Regression

A problem with PCR is that the PC's may not be good predictors for a response variable.

Partial Least Squares, on the other hand, is seeking PC's that

- have maximum *covariance* between predictors and the response
- are orthogonal to each other
- search more directly for y-relevant information for prediction

An example

Let's sketch a situation were PLS will find other latent components than PCA

Fitting the PLSR model in RStudio

```
MVRModel.8 <- plsr(Absorbance ~ ., data=NIRdata, scale=FALSE, ncomp=15)
summary(MVRModel.8)
```

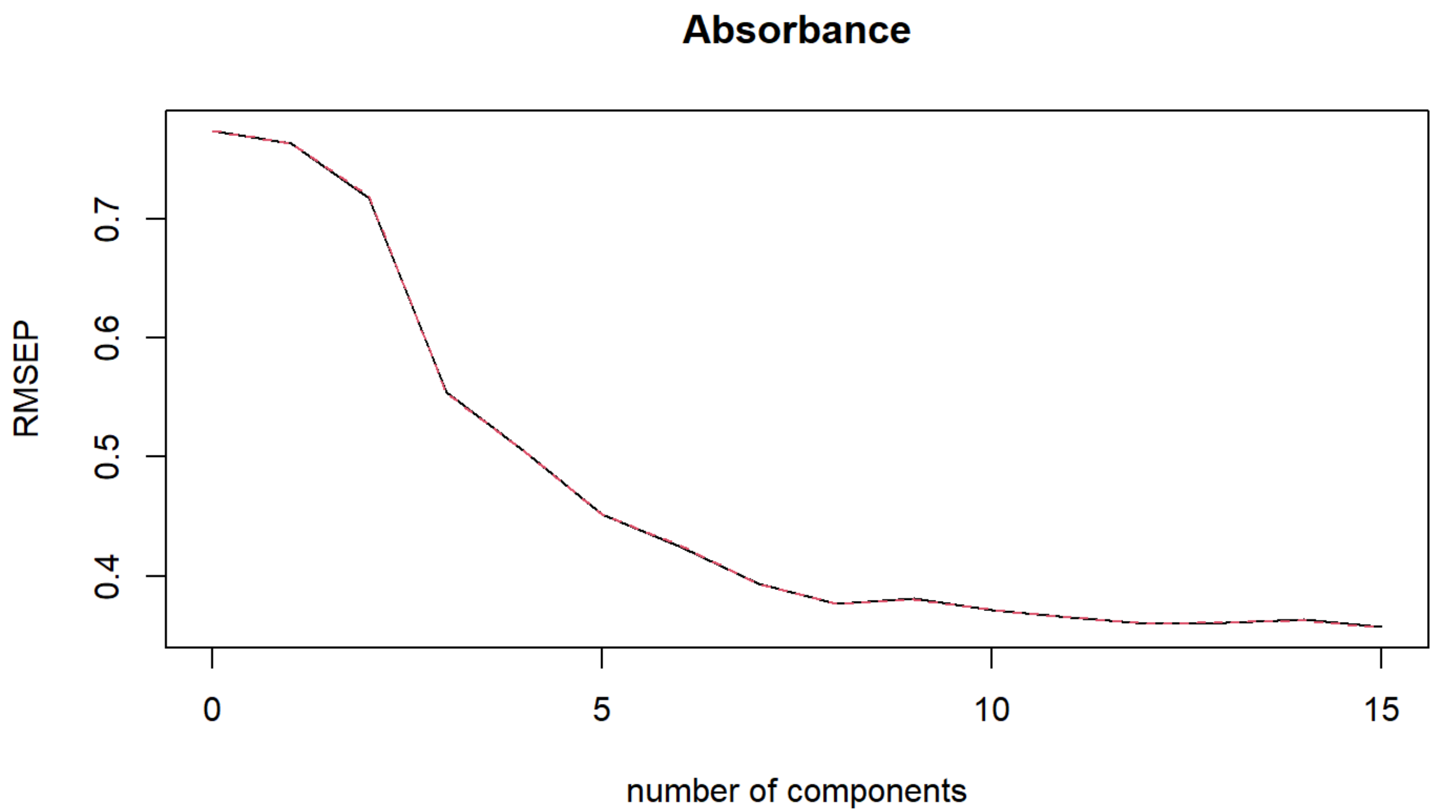
Summary

```
## Data:    X dimension: 100 700
##  Y dimension: 100 1
## Fit method: kernelpls
## Number of components considered: 15
## TRAINING: % variance explained
##             1 comps  2 comps  3 comps  4 comps  5 comps  6 comps  7 comps
## X            45.489    95.56    97.74    98.35    98.76    99.53    99.71
## Absorbance    9.735    17.66    53.15    62.68    72.32    76.26    80.74
##             8 comps  9 comps  10 comps  11 comps  12 comps  13 comps  14 comps
## X             99.76    99.80     99.85     99.89     99.91     99.94     99.95
## Absorbance    83.11    85.33     86.83     87.75     88.68     89.11     90.35
##             15 comps
## X              99.97
## Absorbance     90.71
```

Running LOO-CV and RMSEP plot

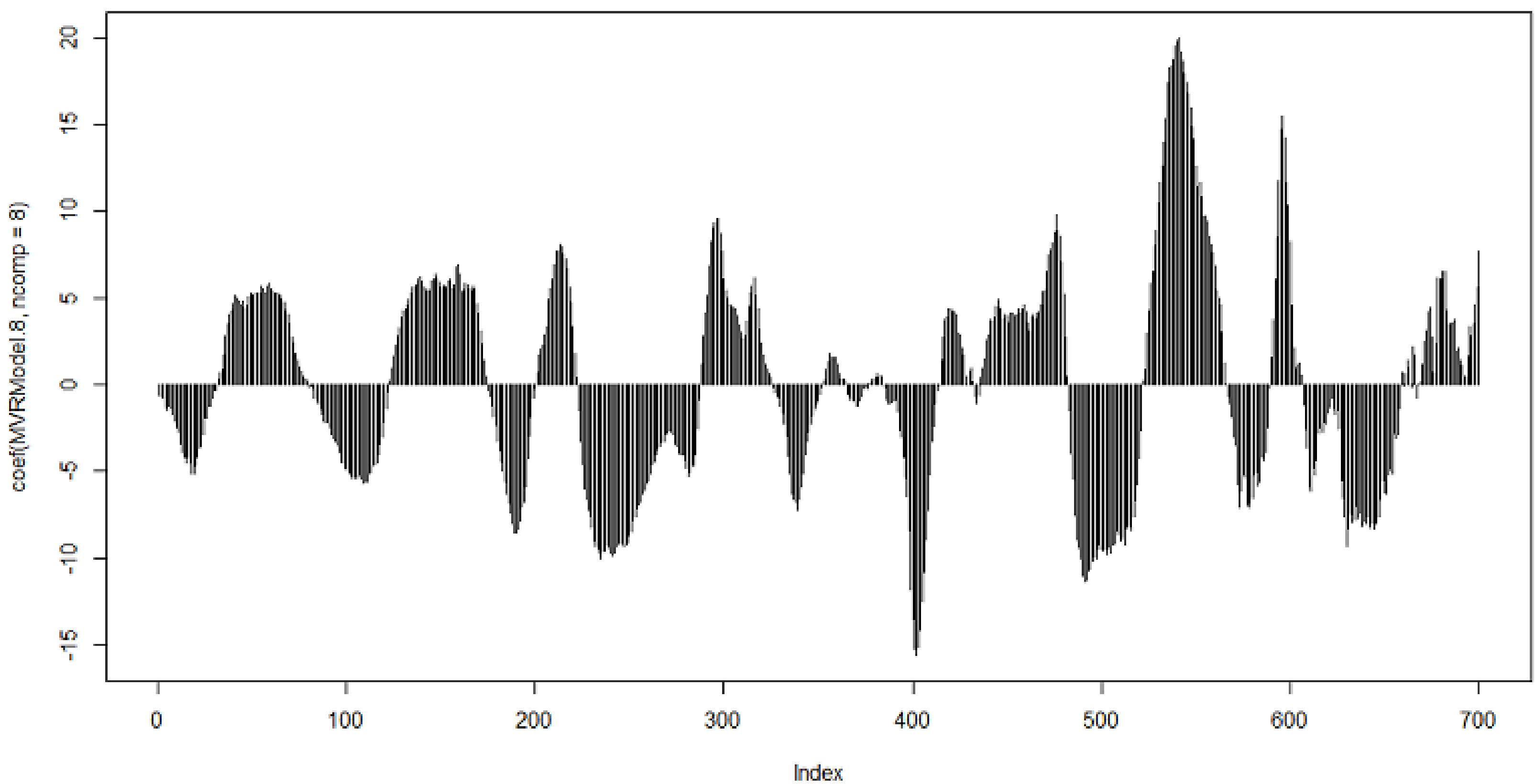
```
MVRModel.8 <- plsr(Absorbance ~ ., data=NIRdata, scale=FALSE, ncomp=15, validation='LO
plot(RMSEP(MVRModel.8))
```

The RMSEP plot



The regression coefficients plotted

- Here are the coefficients based on 8 PLS PC's



Significance testing in PLSR

- No analytical expression for standard errors of estimates!
- No estimate of noise variance!
- Testing based on something called Jack-knifing

Jack-knifing

- Leave-one-out method
- A estimate the variance of the regression coefficients is computed from the saved estimated coefficients from the n LOO-estimations.
- Readily computed in the `pls` package using the `jack.test` function.

$$Var(\hat{\beta}) = \frac{n-1}{n} \sum_{i=1}^{n} (\hat{\beta}_{(-i)} - \hat{\beta})^2$$

Jack-knifing in RStudio

```
MVRModel.8 <- plsr(Absorbance ~ ., data=NIRdata, scale=FALSE, ncomp=15, validation='LO
j.test <- jack.test(MVRModel.8, ncomp = 8, use.mean = FALSE)
print(j.test)
```